

Gigler Orca

AI gig orchestration that turns text, email, and voice into completed work — with a unique email address for every gig.

Google for Startups AI Agents Challenge · Track 2 — Optimize Existing Agents

A completed gig is the product — not a chat transcript.

Users send Gigler an outcome they want over the channels they already use — **SMS, email, or voice**. **Orca**, Gigler's orchestration layer, decomposes the request into steps, routes each step to the right model, agent, tool, or platform, tracks state, pauses for human approval only when needed, and delivers a real artifact through a shareable link.

THE INPUT

Channels people already live in

No app to install, no dashboard to learn. Text it, email it, or call it. Each gig even gets its own email address.

THE BRAIN

Orca orchestration

Plan → route → execute → verify. Picks the cheapest capable tool first; escalates to real browsers, sandboxes, or a human only when required.

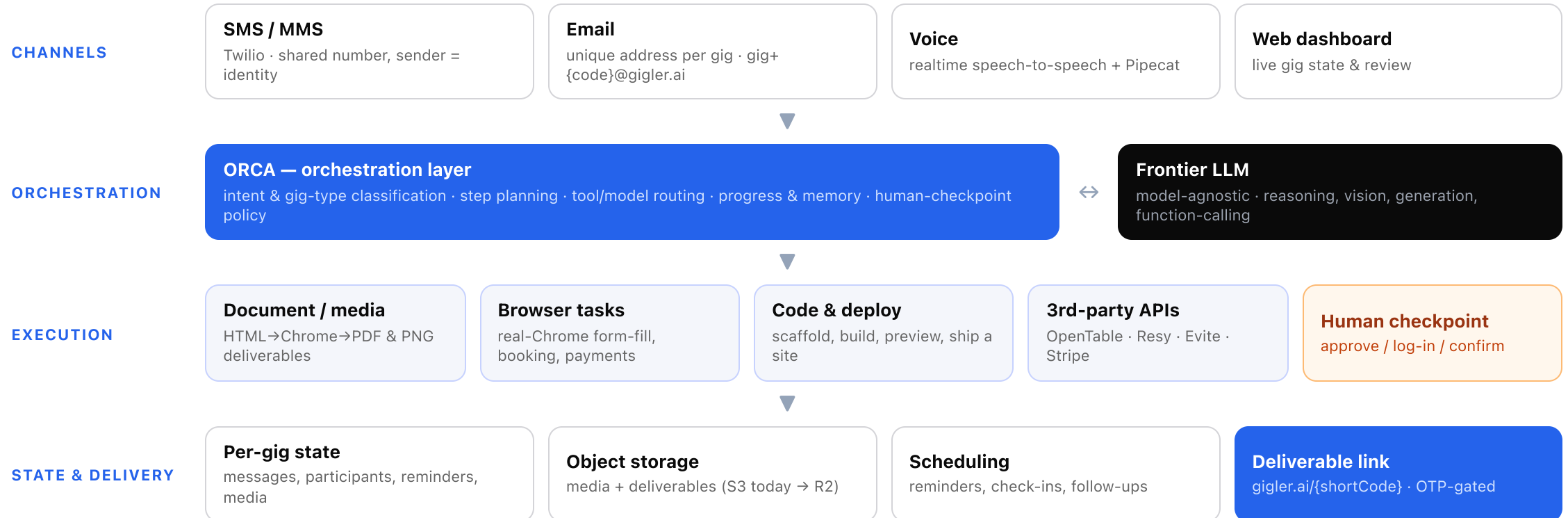
THE OUTPUT

Deliverables, not answers

A deployed site, a designed PDF, a submitted form, a booked reservation — delivered as a [gigler.ai/abc123](#) link.

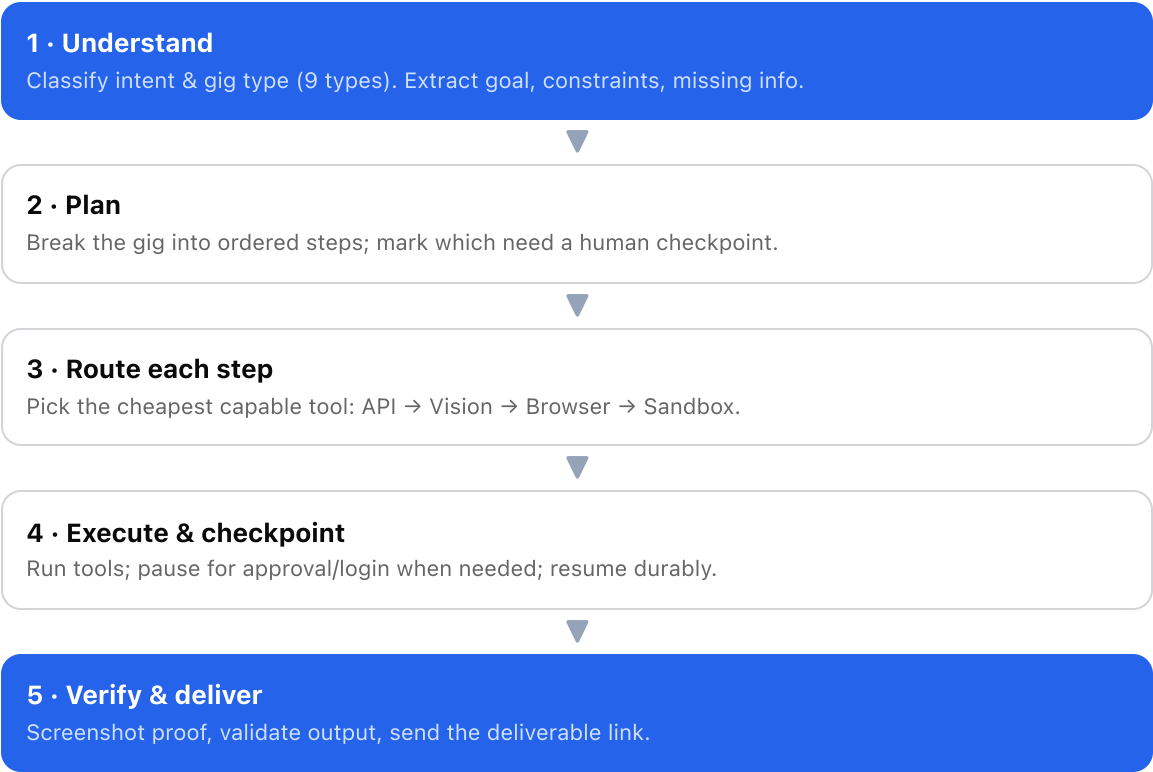
This document describes the current production system (AWS) and the email-native evolution on Cloudflare that hardens Gigler for production scale — the Track 2 "optimize existing agents" story.

From a single request to a delivered gig



Identity is always the **sender** (phone or From address), never the destination — so one shared number and one domain serve thousands of users, each landing in their own gig.

The Orca loop: plan, route, execute, verify



Tool-selection priority (easiest path first)

NEED	ORCA CHOOSES
Structured data exists	Direct API
Read a doc / receipt / screenshot	Model vision
Login, SPA, anti-bot, CSRF	Real browser + HITL
Build / run / deploy code	Sandbox container
Designed document or card	HTML→Chrome→PDF/PNG

Why this is orchestration, not a chatbot

- Multi-step plans with state that survives restarts
- Routes across **models, agents, tools, and platforms**
- Human-in-the-loop only at the moments that need it
- Gets smarter at routing as more gigs complete

Every gig is an agent with its own email address

Email is the most universal interface on earth. Using Cloudflare's Email Service + Agents SDK, each gig gets a unique sub-address that routes inbound mail straight to that gig's agent instance. The inbox becomes the agent's memory.

Inbound email → gig+1a2b3c@gigler.ai

Cloudflare Email Routing receives all *@gigler.ai



routeAgentEmail() + address resolver

sub-address maps to the exact gig agent instance



GigAgent.onEmail()

parse, persist to state, kick off Orca work, reply / escalate async



env.EMAIL.send() — async reply

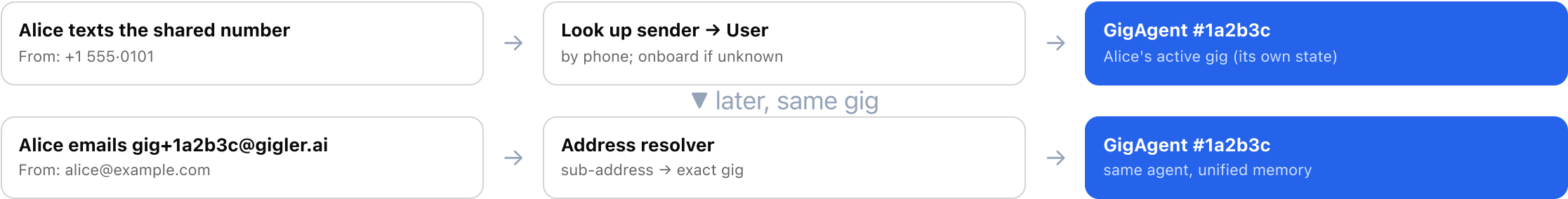
HMAC-signed reply routing back to the same agent

```
export class GigAgent extends Agent { async onEmail(email: AgentEmail) { const msg = await PostalMime.parse(await email.getRaw()); // inbox = memory; persists in Durable Object this.setState({ ...this.state, lastFrom: email.from }); // hand the request to Orca, run async await this.runFiber("orca", () => orca.handle(msg)); } } export default { async email(message, env) { await routeAgentEmail(message, env, { resolver: createAddressBasedEmailResolver("GigAgent"), }); }, };
```

- **No inbox provisioning** — the address itself is the route
- **Asynchronous work** — receive now, process for an hour, reply when done
- **Secure replies** — HMAC-SHA256 headers prevent reply hijacking
- **Same agent, every channel** — SMS, voice & email share one gig state

One identity across every channel, one agent per gig

The system identifies users by **who sent the message** (phone or email From), not by where they sent it. A user can start a gig by text and add details by email — both land in the same gig.



DESTINATION PATTERN	ROUTING BEHAVIOR
<code>gig+{shortCode}@gigler.ai</code>	Address-based resolver → that specific gig agent. Used for replies to per-gig emails.
<code>gigs@gigler.ai</code>	Generic intake → look up sender's From → route to their active gig or spawn a new one.
<code>admin@ · support@ · notifications@</code>	Routed to a single global Admin Inbox agent (team mail, not a user gig).

Multi-tenant safety: 1,000 people emailing **gigs@gigler.ai** land in 1,000 isolated gig agents. Channels are merged only after explicit, verified linking — never silently.

Per-gig agent state + cross-tenant index

One agent (Durable Object) per gig

Each gig is a physically isolated stateful actor with its own embedded SQLite, alarms, and live WebSocket to the dashboard.

- **messages** — full conversation across all channels
- **participants** — members & roles (group gigs)
- **reminders** — durable alarms for follow-ups
- **media** — pointers to R2 objects
- **checkpoints** — durable-execution recovery state

Cross-tenant index (D1 / relational)

A small relational store for everything that must be queried across gigs and users.

- **User** — phone + email identity, plan, billing
- **Gig** — shortCode, owner, status, title
- **Deliverable** — shortCode → R2 key (link lookup)
- **UserIntegration** — encrypted third-party tokens

CONCERN	TODAY (AWS)	HARDENED (CLOUDFLARE)
Per-gig live state	DynamoDB shared tables	Durable Object SQLite — physical isolation per gig
Cross-gig queries / billing	DynamoDB + GSIs	D1 (managed SQLite), indexed
Media & deliverables	S3 + CloudFront	R2 — zero egress, S3-compatible
Reminders	EventBridge scan every 5 min	Per-gig durable alarms (no polling)
Dashboard updates	Polling	Live WebSocket state sync

The tools Orca routes work to

DOCUMENTS & MEDIA

HTML → Chrome → PDF / PNG

Agent writes designed HTML, headless Chrome prints a pitch deck, report, invoice, menu, or MMS card. Source HTML stored for one-line iteration ("make it blue").

BROWSER TASKS

Real Chrome + HITL

Form-fill, bookings, bill pay, applications. Dismisses cookie walls, handles SPAs, captures proof screenshots. Hands off to a human for logins & CAPTCHAs.

CODE & DEPLOY

Sandbox containers

Persistent containers scaffold, run, and deploy a site. Live preview URL streamed to the dashboard. Credentials injected at the egress proxy — agent never sees them.

VISION & REASONING

Frontier LLM (model-agnostic)

Reads PDFs, receipts, and screenshots; drafts copy; classifies intent; calls functions. The reasoning core behind every gig — routed to the best-fit model via a gateway.

THIRD-PARTY PLATFORMS

Adapter pattern

OpenTable, Resy, Evite, Stripe behind a uniform search / execute / confirm / cancel interface. Always confirms with the user before finalizing.

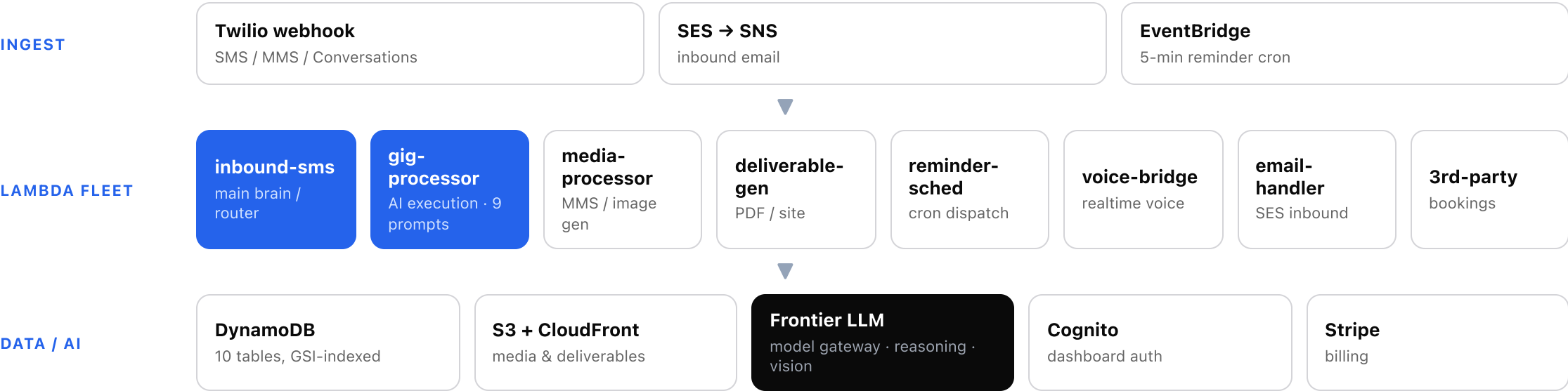
SCHEDULING & VOICE

Reminders + realtime voice

Durable reminders, wake-up calls, check-ins. Real-time speech-to-speech voice for hands-free gig updates.

Every action that touches the outside world is captured (screenshot + metadata) as irrefutable proof of work, stored and searchable from the dashboard.

Today: the production agent on AWS



Stack: AWS Amplify Gen 2 · Next.js 16 · Twilio · model gateway (provider-agnostic LLMs) · Stripe. Proven end-to-end (real gigs, real deliverables) — the foundation we are now optimizing for reliability and scale.

From a working agent to a production-grade one.

The same orchestration brain, hardened: email-native gigs, per-gig isolation, durable execution, and disciplined human-in-the-loop.

What we hardened, and why it matters

DIMENSION	BEFORE (WORKS, BUT BRITTLE AT SCALE)	AFTER (PRODUCTION-GRADE)
Email channel	SES + SNS + Lambda + manual MIME storage	Each gig = an email-native agent (onEmail), unique address, async reply
Gig isolation	Shared tables, logical partitioning	One Durable Object per gig — physical multi-tenant isolation
Long multi-step gigs	15-min Lambda cap; state lost on timeout	Durable execution survives eviction; resumes from last checkpoint
Human-in-the-loop	Ad-hoc SMS confirmation	First-class waitForApproval() with escalation reminders
Reminders	Cron scans the table every 5 min	Per-gig durable alarms — event-driven, no scanning
Deliverable cost	S3 egress per download	R2 zero-egress object storage
Dashboard freshness	Polling	Live WebSocket state sync

Reliability & edge cases are treated as an engineering discipline: mail-loop guards (RFC 3834), secure reply routing, idle-browser cost caps, pre-flight form validation, and proof-of-work capture for every external action — derived from real field tests (e.g. a full agent-driven accelerator application run).

Model-agnostic orchestration, aligned with ADK & MCP

AI across the gig lifecycle

- **Intent & gig-type classification** on every inbound message
- **Multi-step reasoning & planning** inside the Orca loop
- **Vision** — reads PDFs, receipts, screenshots, uploaded photos
- **Generation** — drafts copy, designed HTML, and images
- **Function-calling** — the tool interface Orca routes through
- **Realtime voice** — speech-to-speech for hands-free gigs

Agent patterns & portability

- **MCP** — tools (email, browser, code) exposed as Model Context Protocol servers Orca can securely call
- **ADK-aligned** — declarative agent + tool definitions; Orca maps cleanly onto the Agent Development Kit pattern
- **Model gateway** — a thin gateway routes each step to the best-fit frontier model; no lock-in to any single lab or provider
- **Google Cloud ready** — credits applied to evaluation, optimization, and a path toward Marketplace listing

Orca is not "another chat model." It is an orchestration layer that selects and coordinates the right model, agent, tool, and platform per step — provider-agnostic by design — and improves its routing as more gigs complete.

Production hardening & safety

DURABLE EXECUTION

Nothing gets lost

Long gigs checkpoint and resume after eviction, timeouts, or a closed laptop — the agent picks up from the last completed step.

HUMAN-IN-THE-LOOP

Control where it counts

Logins, payments, irreversible submits, and pre-fill confirmations pause for explicit approval with escalation reminders.

EMAIL SAFETY

No loops, no hijacks

RFC 3834 auto-reply guards prevent mail loops; HMAC-signed reply routing prevents header forgery to other agents.

CREDENTIAL ISOLATION

Secrets stay secret

Third-party tokens injected at the egress proxy; the model and sandbox code never see raw credentials.

COST GUARDRAILS

No runaway agents

Per-gig browser-minute caps, idle-session shutdown, and plan-tier budgets enforced before any expensive tool launches.

PROOF OF WORK

Verifiable outcomes

Every external action stores a confirmation screenshot + metadata, searchable from the dashboard — trust by evidence.

How this maps to the challenge

CRITERION	WHERE GIGLER DELIVERS
Technical (30%)	Multi-step orchestration, durable execution, tool/model routing, real deliverables
Business (30%)	Users want outcomes, not tool menus; Gigler turns fragmented AI into completed work
Innovation (20%)	Orca routing layer + a unique email address per gig across SMS / email / voice
Demo (20%)	One request → orchestrated steps → a shareable, real deliverable

Required assets

- **Code** — repository (Next.js + agents + tools)
- **Video** — 2–3 min, one complete gig end-to-end
- **Architecture diagram** — hosted at gigler.ai/architecture
- **Testing access** — text/email the demo gig and get a deliverable link back

Suggested demo gig

"Build me a one-page site for my dog-walking business and send me the link." Orca plans, scaffolds in a sandbox, screenshots the result, and replies with a live URL + a designed one-pager PDF — all from a single text.

Send the request. Gigler delivers the **work**.

An AI gig-orchestration platform where every gig is an agent — reachable by text, email, or voice — that plans, routes, and delivers real outcomes.